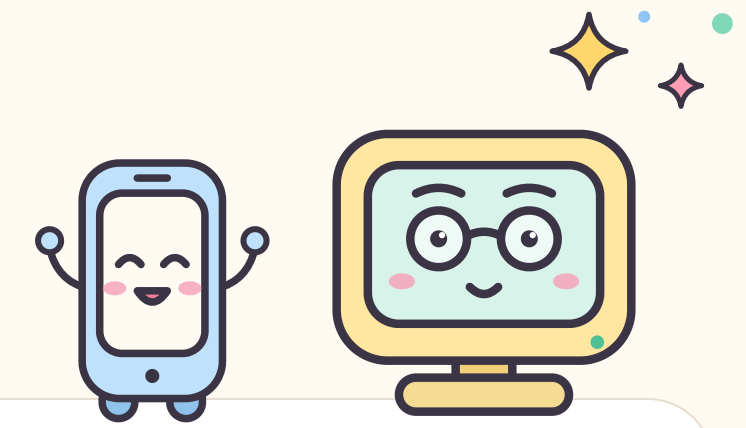




第 13 集 · 约 26 分钟

服务端⑥：加密信道与 RPC

正文一个字节还没读，就知道该不该拒绝

13

从零写出 LanShare

这一集你会学到

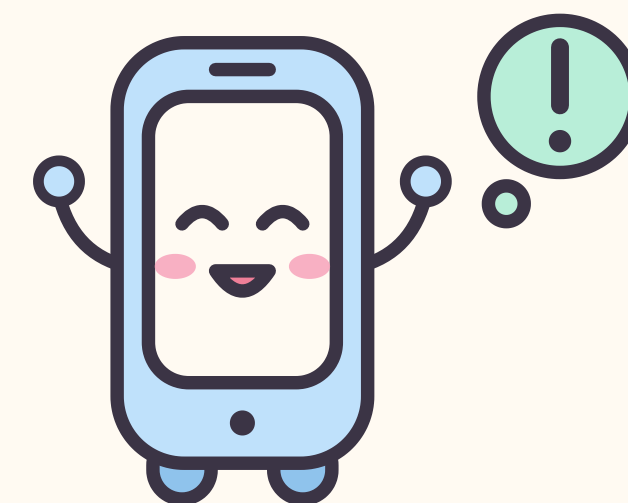
1. 先验请求头，再读正文：into_parts 把请求拆开（审查 I-3）
2. 名额：每个 IP 一份（复验 N-2），全局一个信号量；结构体里放引用要写生命周期
3. 两次加锁防并发重放：锁外解密，锁里再查一次再登记
4. 切片模式匹配：按路径分发；RPC 的 op 表和“只认本机”的操作
5. spawn_blocking：阻塞的文件操作交给专门的线程



先想一个问题

一个请求声明正文有 1 MiB，请求头里的会话号是瞎编的。老电脑应该先把正文读进来，还是先看请求头？

先看请求头。会话不存在，正文一个字节都不读，直接 401。





提取器替你做了事，也替你定了顺序

第一版

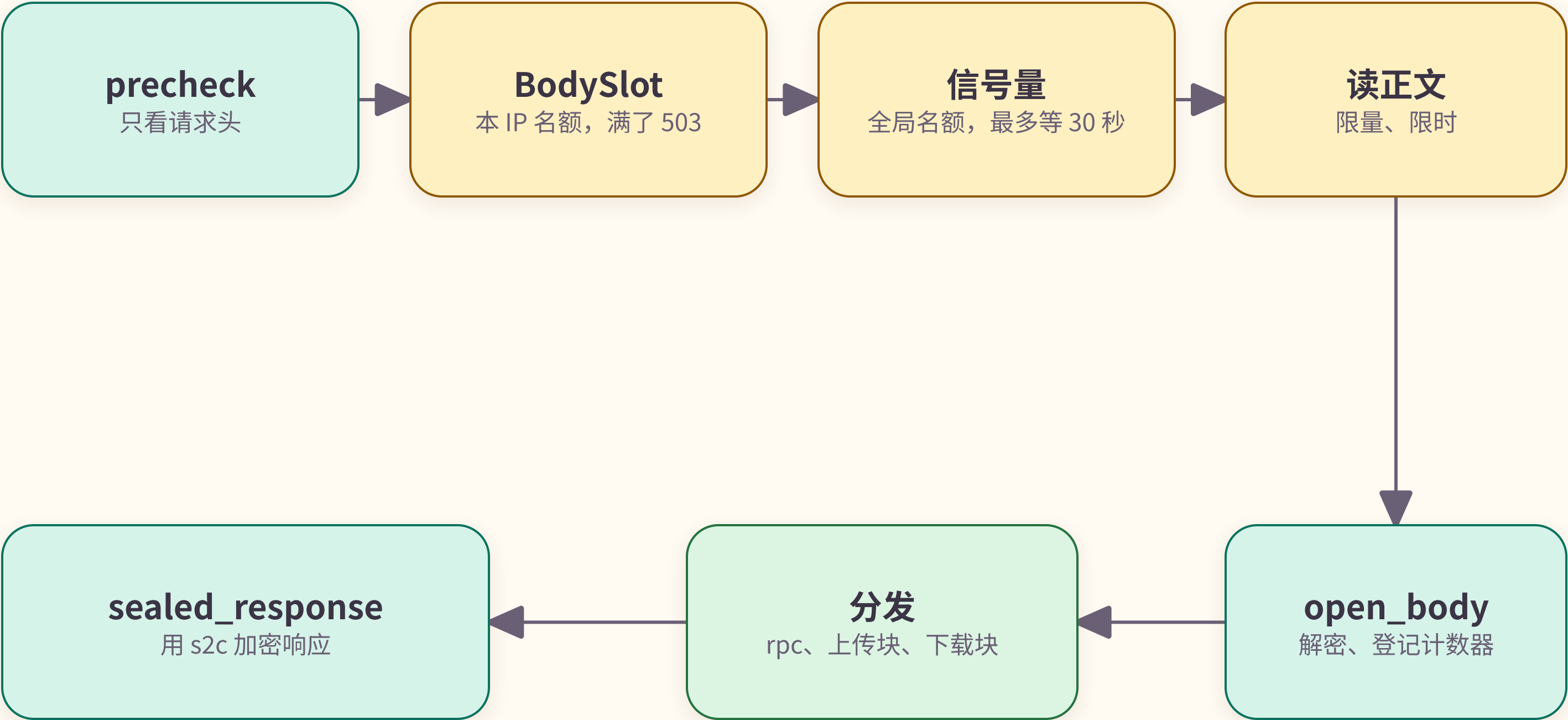
```
1 pub(crate) async fn sealed(  
2     state: St,  
3     ConnectInfo(peer): ConnectInfo<SocketAddr>,  
4     headers: HeaderMap,  
5     uri: Uri,  
6     body: Bytes,           // ← 问题在这里  
7 ) -> Response {
```

现在

```
1 pub(crate) async fn sealed(state: St, ConnectInfo(peer):  
    ConnectInfo<SocketAddr>, request: Request) -> Response {  
2     let st = state.0;  
3     let (parts, body) = request.into_parts(); // 正文还没读  
4     let path = parts.uri.path().to_string();  
5     let (cid, ctr, keys) = match precheck(&st, &parts.headers, &path) {
```



一个加密请求的七道工序





channel.rs: 开头

channel.rs crates/lanshare/src/server/ 第 5-24 行 • 共 24 行

```
5
6 use std::net::SocketAddr;
7
8 use axum::extract::{ConnectInfo, Request};
9 use axum::http::{HeaderMap, HeaderValue, StatusCode, header};
10 use axum::response::{IntoResponse, Response};
11 use lanshare_proto::{self as proto, CHUNK, ChannelKeys, MAX_RPC, TAG_LEN};
12 临时
13 use super::{St, rpc}; 临时
14
15 pub(crate) struct Opened {
16     pub(crate) cid: [u8; 16],
17     pub(crate) ctr: u64,
18     pub(crate) plain: Vec<u8>,
19     s2c: [u8; 32],
20 }
21
22 fn header_str<'a>(headers: &'a HeaderMap, name: &str) -> Option<&'a str> {
23     headers.get(name).and_then(|v| v.to_str().ok())
24 }
```



channel.rs: precheck, 只看请求头

channel.rs
crates/lanshare/src/server/
第 32-51 行 • 共 51 行

```

32     let declared = header_str(headers, "content-length").and_then(|s| s.parse::<usize>().ok());
33     if declared.is_some_and(|n| n > body_limit(path)) {
34         return Err(StatusCode::PAYLOAD_TOO_LARGE);
35     }
36     let cid: Option<[u8; 16]> =
37         header_str(headers, "x-ls-sid").and_then(|s| hex::decode(s).ok()).and_then(|v| v.try_into().ok());
38     let ctr = header_str(headers, "x-ls-ctr").and_then(|s| s.parse::<u64>().ok());
39     let (Some(cid), Some(ctr)) = (cid, ctr) else {
40         return Err(StatusCode::BAD_REQUEST);
41     };
42     // 第一次加锁：取钥匙、预检计数器（不登记）。解密在锁外做，不拖慢其他设备。
43     let mut sessions = st.sessions.lock().expect("锁不会中毒");
44     let Some(session) = sessions.get(&cid) else {
45         return Err(StatusCode::UNAUTHORIZED);
46     };
47     if !session.window.check(ctr) {
48         return Err(StatusCode::FORBIDDEN);
49     }
50     Ok((cid, ctr, session.keys.clone()))
51 }

```



为什么要锁两次

● 第一次加锁：取钥匙、预检计数器，不登记

● 锁外解密

解密 1 MiB 大约 1 毫秒，锁着全局会话表做，所有设备都得排队

● 第二次加锁：再查一次，再登记

两个一模一样的重放请求同时到，都能过第一次检查、都能解密成功



channel.rs: 读正文的名额

channel.rs crates/lanshare/src/server/

第 65–84 行 • 共 84 行

```
65         let n = counts.entry(ip).or_default();
66         if *n >= st.max_bodies_per_ip {
67             return None;
68         }
69         *n += 1;
70         Some(BodySlot { st, ip })
71     }
72 }
73
74 impl Drop for BodySlot<'_> {
75     fn drop(&mut self) {
76         let mut counts = self.st.bodies_per_ip.lock().unwrap_or_else(|e| e.into_inner());
77         if let Some(n) = counts.get_mut(&self.ip) {
78             *n -= 1;
79             if *n == 0 {
80                 counts.remove(&self.ip);
81             }
82         }
83     }
84 }
```



channel.rs: 解密与回信

channel.rs
crates/lanshare/src/server/
第 91-110 行 • 共 110 行

```

91 fn open_body(st: &super::AppState, cid: [u8; 16], ctr: u64, keys: &ChannelKeys, path: &str, body: &[u8]) -> Result<Opened, StatusCode> {
92     let plain = proto::open(&keys.c2s, ctr, &proto::req_aad(path), body).map_err(|_| StatusCode::FORBIDDEN)?;
93     // 第二次加锁: 再查一次再登记, 两个相同的重放请求并发到达时只有一个能通过。
94     let mut sessions = st.sessions.lock().expect("锁不会中毒");
95     let Some(session) = sessions.get(&cid) else {
96         return Err(StatusCode::UNAUTHORIZED);
97     };
98     if !session.window.check(ctr) {
99         return Err(StatusCode::FORBIDDEN);
100     }
101     session.window.commit(ctr);
102     Ok(Opened { cid, ctr, plain, s2c: keys.s2c })
103 }
104
105 fn sealed_response(opened: &Opened, path: &str, payload: &[u8]) -> Response {
106     let body = proto::seal(&opened.s2c, opened.ctr, &proto::resp_aad(path), payload);
107     let mut response = body.into_response();
108     response.headers_mut().insert(header::CONTENT_TYPE, HeaderValue::from_static("application/octet-stream"));
109     response
110 }

```



channel.rs: 入口 sealed



channel.rs

crates/lanshare/src/server/

第 126–145 行 • 共 145 行

```
126     let Ok(Ok(_permit)) = tokio::time::timeout(QUEUE_WAIT, st.bodies.clone().acquire_owned()).await else {
127         return StatusCode::SERVICE_UNAVAILABLE.into_response();
128     };
129     let body = match tokio::time::timeout(st.body_timeout, axum::body::to_bytes(body, body_limit(&path))).await {
130         Ok(Ok(body)) => body,
131         Ok(Err(_)) => return StatusCode::PAYLOAD_TOO_LARGE.into_response(),
132         Err(_) => return StatusCode::REQUEST_TIMEOUT.into_response(),
133     };
134     let opened = match open_body(&st, cid, ctr, &keys, &path, &body) {
135         Ok(opened) => opened,
136         Err(status) => return status.into_response(),
137     };
138     let local = peer.ip().is_loopback();
139     let segments: Vec<&str> = path.trim_start_matches('/').split('/').collect();
140     let payload: Vec<u8> = match segments.as_slice() {
141         ["api", "rpc"] => rpc::handle(&st, &opened.plain, local, opened.cid, peer.ip()).await,
142         _ => rpc::reply_error("not_found"),
143     };
144     sealed_response(&opened, &path, &payload)
145 }
```



按形状匹配：Python 和 Rust

Python 3.10+

```
1 match path.strip("/").split("/"):
2     case ["api", "rpc"]:
3         ...
4     case ["api", "up", upload, index]:
5         ...
6     case _:
7         ...
```

Rust

```
1 match segments.as_slice() {
2     ["api", "rpc"] => ...,
3     ["api", "up", upload, index] => ...,
4     _ => ...,
5 }
```



mod.rs: 读正文的名额和时限

mod.rs

crates/lanshare/src/server/

第 165–184 行 · 共 373 行

```

165         bodies: Arc::new(tokio::sync::Semaphore::new(config.max_inflight_bodies)),
166         max_bodies_per_ip: config.max_bodies_per_ip,
167         bodies_per_ip: Mutex::new(HashMap::new()),
168     }
169 }
170
171 pub fn key(&self) -> [u8; 32] {
172     self.key
173 }
174
175 pub fn server_id(&self) -> &str {
176     &self.server_id
177 }
178
179 pub fn pin(&self) -> String {
180     self.pin.lock().expect("口令锁不会中毒").clone()
181 }
182
183 /// 口令登录是否因为累计失败太多而暂停了。
184 pub fn pin_paused(&self) -> bool {

```



读正文的几道限制

限制	默认值	来历
同时在读的正文（全局）	32 个	审查 I-3：内存占用要有上界
每个 IP 在读的正文	8 个	复验 N-2：一个来源占不光全局名额
排队等全局名额	最多 30 秒	等不到就 503，客户端稍后再试
一个正文必须收完	5 分钟	复验 N-3：原来 60 秒，慢网络传不完；约 3.5 KB/s 也够
正文大小	RPC 64 KiB，块 1 MiB，再加 16 字节标签	声明超长不等正文，直接 413



auth.rs: 按 cid 取会话

auth.rs crates/lanshare/src/server/

第 72-91 行 • 共 307 行

```
72     }
73
74     pub(crate) fn get(&mut self, cid: &[u8; 16]) -> Option<&mut Session> {
75         let session = self.live.get_mut(cid)?;
76         session.last_used = Instant::now();
77         Some(session)
78     }
79 }
80
81 /// 限流：每 IP 的握手失败、每 IP 的口令尝试、全局口令尝试；以及未完成口令尝试的累计数。
82 ///
83 /// 只限速挡不住长期猜口令：每分钟 20 次，一个月就有约一半概率猜中 6 位口令（独立安全审查 I-1）。
84 /// 所以累计未完成的尝试到上限就**暂停**口令登录，只能在本机主界面上重新开启（同时换新口令）。
85 /// 每次开启期间被猜中的概率不超过 上限 / 1,000,000。扫码登录不受影响。
86 #[derive(Default)]
87 pub(crate) struct Guard {
88     hello_failures: HashMap<IpAddr, VecDeque<Instant>>,
89     pake_attempts: HashMap<IpAddr, VecDeque<Instant>>,
90     pake_global: VecDeque<Instant>,
91     outstanding: usize,
```



Cargo.toml: 生成二维码



Cargo.toml

crates/lanshare/

第 11-30 行 • 共 30 行

```
11
12 [dependencies]
13 axum = "0.8"
14 base64 = "0.22"
15 dirs = "6"
16 hex = "0.4"
17 hyper = { version = "1", features = ["http1", "server"] }
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22 rand_core = { version = "0.6", features = ["getrandom"] }
23 serde = { version = "1", features = ["derive"] }
24 serde_json = "1"
25 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
26 tower = { version = "0.5", features = ["util"] }
27 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation", "Win32_Storage_FileSystem"] } 临时
28
29 [dev-dependencies]
30 tempfile = "3"
```



rpc.rs: 开头和工具

● ● ● rpc.rs crates/lanshare/src/server/ 第 8-27 行 • 共 27 行

```
8 use super::AppState; 临时
9
10 pub(crate) fn reply(value: Value) -> Vec<u8> {
11     value.to_string().into_bytes()
12 }
13
14 pub(crate) fn reply_error(code: &str) -> Vec<u8> {
15     reply(json!({ "ok": false, "error": code }))
16 }
17
18 fn qr_svg(url: &str) -> String {
19     match qrcode::QrCode::new(url.as_bytes()) {
20         Ok(code) => code
21             .render::
```



rpc.rs: 按 op 分发

rpc.rs
crates/lanshare/src/server/
第 29-48 行 • 共 48 行

```

29 pub(crate) async fn handle(st: &Arc<AppState>, plain: &[u8], local: bool, cid: [u8; 16], peer: IpAddr) -> Vec<u8> {
30     let _ = (cid, peer); // 临时: 这两个参数第 14 集的上传才用到 临时
31     let Ok(request) = serde_json::from_slice::<Value>(plain) else {
32         return reply_error("bad_request");
33     };
34     match request["op"].as_str().unwrap_or("") {
35         "info" => info(st, local),
36         "list" => list(st).await,
37         "texts" => reply(json!({ "ok": true, "texts": st.texts.list() })),
38         "text_add" => match request["text"].as_str().map(|t| st.texts.add(t)) {
39             Some(Ok(item)) => reply(json!({ "ok": true, "item": item })),
40             Some(Err(code)) => reply_error(code),
41             None => reply_error("bad_request"),
42         },
43         "reveal" => reveal(st, &request, local).await,
44         "open_folder" => open_folder(st, local),
45         "pin_resume" => pin_resume(st, local),
46         _ => reply_error("unknown_op"),
47     }
48 }

```



rpc.rs: info 和 list

rpc.rs crates/lanshare/src/server/

第 60-79 行 • 共 79 行

```
60         "key": st.key_fragment(),
61         "local": local,
62         "chunk": lanshare_proto::CHUNK,
63     });
64     if local {
65         info["folder"] = json!(st.share_dir().display().to_string()); // 本机路径只告诉本机
66     }
67     reply(info)
68 }
69
70 async fn list(st: &Arc<AppState>) -> Vec<u8> {
71     let st2 = st.clone();
72     let files = tokio::task::spawn_blocking(move || st2.storage.list()).await.unwrap_or_default();
73     let mut ids = st.files.lock().expect("锁不会中毒");
74     let files: Vec<Value> = files
75         .into_iter()
76         .map(|f| json!({ "id": ids.id_for(&f.name), "name": f.name, "size": f.size, "mtime": f.mtime }))
77         .collect();
78     reply(json!({ "ok": true, "files": files }))
79 }
```



rpc.rs: 只认本机的三个操作

rpc.rs crates/lanshare/src/server/

第 97-116 行 • 共 116 行

```
97 fn pin_resume(st: &AppState, local: bool) -> Vec<u8> {
98     if !local {
99         return reply_error("only_local");
100     }
101     let pin = st.resume_pin();
102     st.log("已重新开启口令登录，并换了新口令");
103     reply(json!({ "ok": true, "pin": pin }))
104 }
105
106 fn open_folder(st: &AppState, local: bool) -> Vec<u8> {
107     if !local {
108         return reply_error("only_local");
109     }
110     let Some(desktop) = st.desktop.clone() else { return reply_error("unavailable") };
111     if st.storage.ensure_root().is_err() {
112         return reply_error("folder_unavailable");
113     }
114     desktop.open_folder(st.share_dir());
115     reply(json!({ "ok": true }))
116 }
```



mod.rs: 文本、文件 id 和 “重新开启”

● ● ●

mod.rs

crates/lanshare/src/server/

第 182-201 行 • 共 404 行

```
182     files: Mutex::new(FileIds::default()),
183     desktop: config.desktop,
184     log: config.log,
185     max_sessions: config.max_sessions,
186     per_ip_limit: config.per_ip_limit,
187     global_limit: config.global_limit,
188     pin_fail_limit: config.pin_fail_limit,
189     body_timeout: config.body_timeout,
190     idle_timeout: config.idle_timeout,
191     max_connections: config.max_connections,
192     max_local_connections: config.max_local_connections,
193     max_connections_per_ip: config.max_connections_per_ip,
194     header_timeout: config.header_timeout,
195     handshake_timeout: config.handshake_timeout,
196     bodies: Arc::new(tokio::sync::Semaphore::new(config.max_inflight_bodies)),
197     max_bodies_per_ip: config.max_bodies_per_ip,
198     bodies_per_ip: Mutex::new(HashMap::new()),
199 }
200 }
201
```



“重新开启口令登录”

mod.rs crates/lanshare/src/server/

第 337-356 行 · 共 417 行

```

337 // 加密接口自己控制正文读取：先验请求头，再在限量、限时的情况下读正文
338 .route("/api/rpc", post(channel::sealed))
339 .route("/api/up/{upload}/{index}", post(channel::sealed))
340 .route("/api/down/{file}/{index}", post(channel::sealed))
341 .layer(middleware::from_fn(check_origin))
342 .layer(middleware::from_fn(security_headers))
343 .layer(middleware::from_fn(check_host))
344 .with_state(state)
345 }
346
347 /// 在已绑定的监听器上提供服务，直到出错。
348 pub async fn serve(listener: tokio::net::TcpListener, state: Arc<AppState>) -> std::io::Result<()> {
349     serve_with_shutdown(listener, state, std::future::pending()).await
350 }
351
352 /// 同上，`shutdown` 完成时停止接受新连接，等进行中的请求结束（最多 3 秒；托盘“退出”时用）。
353 ///
354 /// 没用 `axum::serve`：它不给 hyper 配计时器，读请求头没有时限（独立安全审查 I-3、复验 N-1）。
355 /// 这里自己接收连接：先过准入（每 IP 上限、本机单独一池），再交给 hyper，配上计时器和请求头时限。
356 pub async fn serve_with_shutdown(

```



auth.rs: Guard 恢复

● ● ●

auth.rs crates/lanshare/src/server/

第 98-117 行 • 共 312 行

```
98     }
99
100     pub(crate) fn resume(&mut self) {
101         self.paused = false;
102         self.outstanding = 0;
103     }
104 }
105
106 fn recent(q: &mut VecDeque<Instant>, now: Instant) -> usize {
107     while q.front().is_some_and(|t| now.duration_since(*t) > WINDOW) {
108         q.pop_front();
109     }
110     q.len()
111 }
112
113 fn parse_cid(s: &str) -> Option<[u8; 16]> {
114     hex::decode(s).ok()?.try_into().ok()
115 }
116
117 /// 请求体解析成 JSON；格式不对、字段缺失一律当作 400，不暴露解析细节。
```



加密信道和 RPC 写好了

```
lanshare/
├── crates/
│   └── lanshare/
│       └── src/
│           ├── server/
│           │   ├── assets.rs
│           │   ├── auth.rs  这一集写的
│           │   ├── channel.rs  这一集写的
│           │   ├── conn.rs
│           │   ├── mod.rs  这一集写的
│           │   └── rpc.rs  这一集写的
│           └── cli.rs
```

```
├── lib.rs
├── logging.rs
├── main.rs
├── netinfo.rs
├── storage.rs
├── textboard.rs
├── web/  6 个文件
├── Cargo.toml  这一集写的
├── lanshare-proto/  13 个文件
├── lanshare-wasm/  2 个文件
├── web-tests/  2 个文件
└── Cargo.toml
```



编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Updating crates.io index
  Locking 1 package to latest Rust 1.96.0 compatible version
  Adding qrcode v0.14.1
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 11.09s

PS D:\code\lanshare> cargo test -p lanshare --lib
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 4.34s
  Running unittests src\lib.rs (target\debug\deps\lanshare-c9b4a6a3f956eda2.exe)

running 23 tests
..... (省略 23 行, 和上一集一样, 全部 ok)

test result: ok. 23 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.02s
```





不带会话来敲门

```
Windows PowerShell

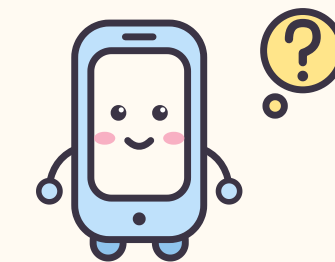
PS D:\code\lanshare> cargo run -p lanshare
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.30s
    Running `target\debug\LanShare.exe`
LanShare 骨架跑起来了, 协议版本: lanshare/2
主界面地址: http://127.0.0.1:8765/#-ZaJexfrhJip9gbz3na-b6LrCkgzedJnEB1RRhyxcQI

PS D:\code\lanshare> curl.exe -s -w "%{http_code}\n" -H "Content-Type: text/plain" -d hi http://127.0.0.1:8765/api/rpc
415

PS D:\code\lanshare>
curl.exe -s -w "%{http_code}\n" -H "Content-Type: application/octet-stream" -d hi http://127.0.0.1:8765/api/rpc
400

PS D:\code\lanshare>
curl.exe -s -w "%{http_code}\n" -X POST -H "Content-Type: application/octet-stream" -H "X-LS-Sid:
00112233445566778899aabbccddeeff" -H "X-LS-Ctr: 1" -H "Content-Length: 65552" http://127.0.0.1:8765/api/rpc
401

PS D:\code\lanshare>
curl.exe -s -w "%{http_code}\n" -X POST -H "Content-Type: application/octet-stream" -H "X-LS-Sid:
00112233445566778899aabbccddeeff" -H "X-LS-Ctr: 1" -H "Content-Length: 65553" http://127.0.0.1:8765/api/rpc
413
```



选择题

两个一模一样的重放请求同时到达。precheck 都通过了，也都解密成功了。是什么保证只有一个被处理？

A precheck 里登记了计数器

B open_body 第二次加锁，再查一次，并在同一把锁里登记

C 解密只能成功一次



选择题

两个一模一样的重放请求同时到达。precheck 都通过了，也都解密成功了。是什么保证只有一个被处理？

A precheck 里登记了计数器

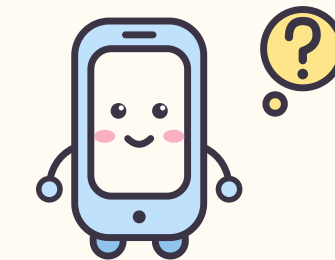
precheck 只检查不登记，还没解密，不知道请求是真是假。

C 解密只能成功一次

同一份密文可以解密任意多次，解密本身挡不住重放。

B open_body 第二次加锁，再查一次，并在同一把锁里登记

第二次加锁时先检查再 commit，两件事在同一把锁里完成。先拿到锁的那个登记了计数器，后来的再查就不通过了。



选择题

sealed 里写的是 `let Some(_mine) = BodySlot::take(...)`。如果改成 `let Some(_) = ...`，会怎样？

A 没区别，都是不用的变量

B 名额刚拿到就还回去，每 IP 的限制失效

C 编译不过



选择题

sealed 里写的是 `let Some(_mine) = BodySlot::take(...)`。如果改成 `let Some(_) = ...`，会怎样？

A 没区别，都是不用的变量

光下划线不绑定值，名额当场被丢弃，Drop 立刻把它还回去。

C 编译不过

两种写法都能编译，这正是它危险的地方。

B 名额刚拿到就还回去，每 IP 的限制失效

光下划线不会把值留住，BodySlot 当场被丢弃，Drop 立刻退还名额。带名字的下划线 mine 会一直持有到函数结束。



选择题

下面哪一种请求，服务端在读正文之前就会拒绝？

A 会话和计数器都对，但正文被篡改了

B 会话号是瞎编的

C 计数器对，正文是合法密文



选择题

下面哪一种请求，服务端在读正文之前就会拒绝？

A 会话和计数器都对，但正文被篡改了

篡改要解密才能发现，那时正文已经读进来了，结果是 403。

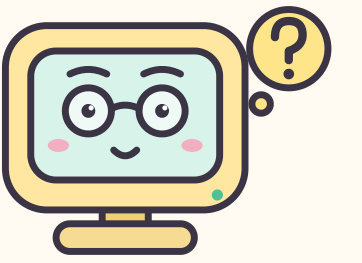
C 计数器对，正文是合法密文

这是正常请求，会读正文、解密、处理。

B 会话号是瞎编的

precheck 只凭请求头就查出会话不存在，直接 401，正文一个字节都不读。这正是审查 I-3 的修法。

想一想



加密接口出了业务错误，比如文件不存在，HTTP 状态码仍然是 200，错误写在加密的 JSON 里。为什么不直接回 404？



想一想

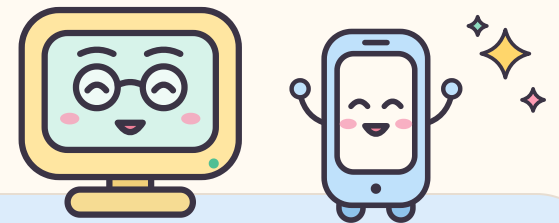
加密接口出了业务错误，比如文件不存在，HTTP 状态码仍然是 200，错误写在加密的 JSON 里。为什么不直接回 404？

状态码是明文，窃听者看得见。如果业务错误用状态码表示，窃听者就能知道“这次下载的文件不存在”“这次上传被拒绝了”，泄露了操作的结果。全部回 200、错误放进密文里，窃听者只能看到“有一个请求、有一个响应”。只有服务端还没能解开请求时的问题，比如会话不存在、计数器不对、太大、解不开，才用明文状态码：这些情况下服务端本来就还不知道请求的内容。



这一集写了什么

- ✓ 先验请求头再读正文：into_parts 拆开，precheck 只看头，不合格一个字节都不读（审查 I-3）
- ✓ 名额：每 IP 一份（复验 N-2）+ 全局信号量；带生命周期的 BodySlot，Drop 自动退还
- ✓ 锁外解密，锁里再查再登记，并发重放只有一个能通过
- ✓ 切片模式匹配按路径分发；RPC 只有一个路径，op 在密文里
- ✓ spawn_blocking 跑阻塞的文件操作；只认本机的操作看对方是不是回环地址



下一集

全课完结，感谢收看